

FastPagePro Studio v2.0

Documento de Validacion Final

Integracion Sanity CMS v3 + Next.js 16
Arquitectura de Plantilla Maestra para Agencias

Proyecto: Sertrade Design

<https://sertrade.vercel.app>

Fecha: 4 de junio, 2026
Version: 1.0.0 - Pre-implementacion

Desarrollado por FastPagePro.com

1. Resumen Ejecutivo

Este documento establece la validacion final y completa antes de iniciar la implementacion de la integracion entre Sanity CMS v3 y el sitio web de Sertrade Design construido con Next.js 16.1 (App Router), Tailwind CSS v4, y Framer Motion 12. La arquitectura disenada no es exclusiva para Sertrade: es una plantilla maestra reutilizable para cualquier proyecto de FastPagePro, cubriendo verticales como construccion, arquitectura, ingenieria, logistica, industria y corporativo general.

Todos los contenidos visibles del sitio seran editables desde el Sanity Studio embebido en /admin, incluyendo textos, imagenes, videos, SEO, Open Graph, hero slides con soporte de video MP4/WebM, proyectos, servicios, testimonios, equipo y estadisticas animadas. El credito del pie de pagina "Desarrollado por FastPagePro.com" permanecera permanentemente hardcodedo y excluido del CMS, Visual Editing y Presentation Tool.

La infraestructura de Sanity ya fue desplegada en el commit a02f484 (Fast Page Pro Studio v2.0), incluyendo la configuracion del Studio, los 10 schemas documentales, el cliente con Stega para Visual Editing, las queries GROQ centralizadas, el sistema defineLive para revalidacion en tiempo real, los API routes de Draft Mode y el overlay de Visual Editing. Sin embargo, los componentes del frontend (HomePage, ProjectsPage, ServicesPage) aun consumen datos hardcodedos y necesitan ser migrados para consultar datos desde Sanity.

La implementacion se dividira en 5 fases secuenciales: primero la infraestructura base (configuracion, schemas, helpers), segundo el cliente y queries, tercero el Studio route handler, cuarto la conexion del layout principal con SanityLive y VisualEditing, y quinto la migracion gradual de cada pagina del frontend (Footer, Servicios, Proyectos, Home) para consumir datos del CMS en lugar de datos estaticos. Cada fase se validara antes de pasar a la siguiente.

2. Arbol Completo de Archivos Nuevos

Los siguientes archivos seran creados o verificados durante la implementacion. Los archivos marcados con (*) ya existen en el commit a02f484 y seran verificados/actualizados. Los demas se crean desde cero.

```
sanity/  
  config.ts (*)  
  lib/  
    constants.ts (*)  
    schema-master.ts (*)  
  schema.ts (*)  
  schemas/  
    index.ts (*)  
    siteSettings.ts (*)  
    heroSlide.ts (*)  
    serviceCategory.ts (*)  
    service.ts (*)  
    project.ts (*)  
    teamMember.ts (*)  
    testimonial.ts (*)  
    partner.ts (*)  
    stat.ts (*)  
  
src/lib/  
  sanity.client.ts (*)
```

```

sanity.queries.ts (*)

src/sanity/
  live.ts (*)

src/app/admin/[...tool]/
  page.tsx (*)

src/app/api/draft-mode/
  enable/route.ts (*)
  disable/route.ts (*)

src/components/
  VisualEditing.tsx (*)

```

Total: 22 archivos relacionados con Sanity. De estos, 21 ya existen en la infraestructura desplegada (commit a02f484) y serán verificados. Los archivos que requieren creación o modificación importante son los componentes del frontend (HomePage, ProjectsPage, ServicesPage, Footer) y el layout.tsx que aun no están conectados al CMS.

3. Archivos Modificados

Los siguientes archivos existentes serán modificados para conectar el CMS al frontend. Cada modificación es incremental y retrocompatible: si el CMS no responde, los componentes mostrarán los datos hardcodeados existentes como fallback.

Archivo	Modificación	Riesgo
src/app/layout.tsx	Envolver children con <SanityLive> + <VisualEditing> para revalidación en tiempo real y overlay de edición inline	Bajo
src/components/HomePage.tsx	Reemplazar datos hardcodeados de hero, stats, servicios, proyectos, pilares y logos por fetch a Sanity queries con fallback	Medio
src/components/ProjectsPage.tsx	Reemplazar 6 proyectos hardcodeados por fetch a query allProjects; mantener galería y video lightbox intactos	Medio
src/components/ServicesPage.tsx	Reemplazar 3 módulos de servicio por fetch a query allServices con subservicios y categorías dinámicas	Medio
src/components/Footer.tsx	Reemplazar datos de contacto, redes y dirección por fetch a siteSettings; MANTENER crédito FastPagePro hardcodeado	Bajo
next.config.ts	Agregar cdn.sanity.io en images.remote para optimización de imágenes desde CDN de Sanity	Bajo

4. Dependencias a Instalar

Las siguientes dependencias son necesarias para Sanity CMS v3. Algunas ya estan en package.json pero se listan para confirmar versiones minimas requeridas.

Paquete	Version	Proposito
next-sanity	9.x	Cliente Sanity optimizado para Next.js App Router. Incluye NextStudio, createClient, defineRouteHandler.
sanity	3.x	Core de Sanity Studio: defineConfig, defineType, defineField, plugins.
@sanity/image-url	1.x	Constructor de URLs optimizadas para imagenes de Sanity CDN con parametros de transformacion.
@sanity/icons	2.x	Iconos oficiales para identificar tipos de documento en el panel del Studio.

Comando de instalacion:

```
bun add next-sanity sanity @sanity/image-url @sanity/icons
```

5. Variables de Entorno Necesarias

Variables obligatorias para desarrollo local (.env.local) y produccion (Vercel dashboard). Las credenciales se obtienen de manage.sanity.io.

Variable	Ejemplo	Descripcion
NEXT_PUBLIC_SANITY_PROJECT_ID	abc123xyz	ID unico del proyecto Sanity. Publica para image URL builder del lado del cliente.
NEXT_PUBLIC_SANITY_DATASET	production	Nombre del dataset. Por defecto "production".
SANITY_API_READ_TOKEN	sk_abc123...	Token solo lectura (CDA). Server-side. Obligatorio para prevenir dataset publico.
SANITY_API_TOKEN_PREVIEW	sk_preview...	Token con permisos de preview. Server-side. Para Draft Mode y Presentation Tool.
NEXT_PUBLIC_SANITY_STUDIO_URL	/admin	URL del Studio embebido. Publica para redirect desde Presentation Tool.
NEXT_PUBLIC_SITE_URL	https://sertrade.vercel.app	URL publica del sitio. Para OG images, sitemaps y metadatos SEO dinamicos.

Archivo .env.local.example:

```
NEXT_PUBLIC_SANITY_PROJECT_ID=tu_project_id
NEXT_PUBLIC_SANITY_DATASET=production
SANITY_API_READ_TOKEN=tu_read_token
SANITY_API_TOKEN_PREVIEW=tu_preview_token
NEXT_PUBLIC_SANITY_STUDIO_URL=/admin
NEXT_PUBLIC_SITE_URL=https://tu-dominio.vercel.app
```

Las variables NEXT_PUBLIC_ son accesibles del lado del cliente. Las demas son exclusivas del servidor y nunca se exponen al navegador. Los tokens de API permiten acceder a los documentos del CMS, incluyendo drafts, por

lo que deben protegerse estrictamente.

6. Schemas Definitivos (10 Tipos de Documento)

Los schemas definen la estructura de documentos en Sanity. Son genericos y reutilizables para cualquier proyecto de FastPagePro, no solo para Sertrade. Cada schema se configura con `defineType` y contiene campos con `defineField`. Los singletons (`siteSettings`, `studioGuide`) solo permiten un documento; los demas soportan multiples instancias.

Schema	Tipo	Campos principales
siteSettings	Singleton	title, description, ogImage, favicon, logo, phone, email, address, socialLinks[], location (geopoint), seoTitle, seoDescription, ogTitle, ogDescription, canonicalUrl
heroSlide	Multiple	title, subtitle, ctaText, ctaLink, backgroundImage, videoMP4 (file), videoWebM (file), posterImage, muted, autoplay, loop, mobileFallbackImage, order
serviceCategory	Multiple	title, slug, description, icon (icon picker), accentColor, image
service	Multiple	title, slug, shortDescription, featuredImage, category (ref), subservices[] (title, description, icon, image), featured, order
project	Multiple	title, slug, client, year, description (Portable Text), featuredImage, gallery[] (max 15), tags[], status (enum), category, videoUrl
teamMember	Multiple	name, slug, role, photo, bio (Portable Text), email, phone, socialLinks[], order
testimonial	Multiple	name, role, company, quote, rating (1-5), avatar, featured
partner	Multiple	name, logo (image), url, order
stat	Multiple	label, value (number), suffix (+/%/K), icon, order, featured
studioGuide	Singleton	title, content (Portable Text), version

6.1 Detalle del Schema heroSlide (Soporte de Video)

El schema heroSlide es el mas complejo porque soporta imagen estatica, video MP4, video WebM, poster image y fallback movil. Cada campo es opcional (excepto title y backgroundImage) lo que permite crear slides solo con imagen, solo con video, o con combinacion de ambos. Los booleanos muted/autoplay/loop controlan el comportamiento del reproductor.

Campo	Tipo Sanity	Req.	Descripcion
title	string	Si	Titulo principal del slide
subtitle	string	No	Subtitulo o descripcion breve
ctaText	string	No	Texto del boton de accion
ctaLink	string	No	URL o slug destino del CTA
backgroundImage	image	Si	Imagen de fondo principal (fallback universal)

Campo	Tipo Sanity	Req.	Descripcion
videoMP4	file (video/mp4)	No	Video MP4 de fondo para escritorio
videoWebM	file (video/webm)	No	Video WebM para navegadores compatibles
posterImage	image	No	Imagen mostrada mientras el video carga
muted	boolean	No	Silenciar video (default: true)
autoplay	boolean	No	Reproducir automaticamente (default: true)
loop	boolean	No	Reproducir en bucle (default: true)
mobileFallbackImage	image	No	Imagen especifica para moviles
order	number	Si	Posicion en el carrusel

7. Queries GROQ Definitivas (10 Queries)

GROQ (Graph-Relational Object Queries) es el lenguaje de consulta de Sanity. Todas las queries estan centralizadas en `src/lib/sanity.queries.ts`. Cada query retorna solo los campos necesarios con proyecciones para imagenes optimizadas y coalesce para valores por defecto.

Query	Proposito	Retorna
siteSettingsQuery	Config global	1 documento singleton con datos de contacto, redes, SEO y OG
allHeroSlidesQuery	Hero carousel	Array ordenado por "order": imagenes, videos, poster y fallback movil
allServicesQuery	Servicios	Array con categoria referenciada, subservicios inline, imagen destacada
allProjectsQuery	Portafolio	Array con galeria (max 15), tags, estado, video URL
allTeamMembersQuery	Equipo	Array ordenado con foto, bio (Portable Text), redes sociales
allTestimonialsQuery	Testimonios	Array filtrado por "featured" con quote, avatar, rating
allPartnersQuery	Partners	Array ordenado con nombre, logo, URL del sitio
allStatsQuery	Estadisticas	Array filtrado por "featured" con valor numerico y sufijo
serviceBySlugQuery	Servicio por slug	1 documento con subservicios y categoria
projectBySlugQuery	Proyecto por slug	1 documento con galeria, descripcion (Portable Text), tags

7.1 Ejemplo: allHeroSlidesQuery

```
*[_type == "heroSlide"] | order(order asc) {
  _id, title, subtitle, ctaText, ctaLink,
  "bgImage": backgroundImage { asset-> { url, metadata { dimensions } } },
```

```
"videoMp4": videoMP4.asset->url,  
"videoWebm": videoWebM.asset->url,  
"poster": posterImage { asset-> { url, metadata { dimensions } } },  
muted, autoplay, loop,  
"mobileImg": mobileFallbackImage { asset-> { url, metadata { dimensions } } },  
order  
}
```


8. Contenido Editable vs. Hardcodeado

8.1 Contenido Editable desde Sanity Studio

Todo el contenido visible sera editable desde /admin sin tocar codigo ni hacer deploys. Los cambios en documentos publicados se reflejan inmediatamente gracias a defineLive que escucha mutaciones via WebSocket y dispara revalidacion del cache ISR de Next.js.

- **Textos:** Todos los textos publicos: titulos, descripciones, subtítulos, CTAs, textos de contacto, direcciones, textos del header y footer (seccion contacto)
- **Imágenes:** Todas las imágenes: hero slides, servicios, galerias de proyectos, fotos de equipo, avatares, logos de partners, iconos, OG image, favicon, logo del sitio
- **Videos:** Videos de hero slides: MP4, WebM, poster image. Video URL en proyectos. Parametros muted, autoplay, loop
- **Hero Video:** Cada slide soporta: imagen + video MP4 + video WebM + poster + fallback movil. Todo editable campo por campo
- **Open Graph:** Titulo OG, descripcion OG, imagen OG, URL canonica desde siteSettings del CMS. Metadata dinamica en layout.tsx
- **SEO:** Title tag, meta description, URL canonica. Obtenido de siteSettings e inyectado en metadata API de Next.js App Router
- **Proyectos:** Titulo, cliente, ano, descripcion (Portable Text), imagen destacada, galeria (15 imgs), tags, estado, categoria, video URL
- **Servicios:** Titulo, slug, descripcion, imagen, categoria, subservicios (titulo + desc + icono + imagen), orden, destacado
- **Testimonios:** Nombre, rol, empresa, quote, rating (1-5), avatar, destacado
- **Equipo:** Nombre, rol, foto, bio (Portable Text), email, telefono, redes sociales, orden
- **Config global:** Nombre empresa, telefono, email, direccion, redes sociales, ubicacion GPS, colores, titulo, descripcion

8.2 Contenido Hardcodeado (Fuera del CMS)

Estos elementos permaneceran permanentemente fuera del CMS, Visual Editing y Presentation Tool por disenio para proteger la integridad de marca y elementos funcionales.

- **"Desarrollado por FastPagePro.com":** String literal en Footer.tsx. Nunca en Sanity, nunca editable en Studio, ignorado por VisualEditing. Atributo de marca inmutable.
- **URL del credito:** href apunta siempre a <https://fastpagepro.com>. No almacenado en CMS ni configurable.
- **Estructura de componentes:** Layout HTML/JSEX de Header, Footer, HomePage. No editable. Solo los datos que consumen son editables.

- **Logica de animaciones:** Animaciones Framer Motion (fadeIn, slideIn, flip cards) hardcodeadas en componentes y CSS. No configurables.
- **Rutas de navegacion:** Paths (/ , /proyectos, /servicios, /admin) hardcodeados en Header. Solo textos de links serian editables en futuro.
- **Libro de Reclamaciones:** Formulario con validacion Zod, envio simulado. No conectado al CMS.

9. Funcionamiento de Visual Editing

Visual Editing permite overlayear un panel transparente sobre el sitio en produccion, para que un editor haga clic en cualquier elemento editable y lo modifique directamente, viendo los cambios en tiempo real. Funciona en tres capas: Stega en el cliente, overlay de VisualEditing, y edicion inline con revalidacion automatica.

- **Capa 1 - Cliente con Stega:** El cliente de Sanity se configura con stega: { enabled: true, studioUrl: "/admin" } en src/lib/sanity.client.ts. Cuando se consulta un documento, Sanity inyecta metadatos invisibles (source maps) en cada campo, conteniendo referencias al documento, campo y proyecto.
- **Capa 2 - Overlay:** El componente VisualEditing.tsx (ya existente) se renderiza en layout.tsx envolviendo el contenido. Detecta si el usuario esta autenticado en el Studio y si los datos contienen Stega. Si ambas condiciones se cumplen, renderiza un overlay con indicadores visuales en cada campo editable.
- **Capa 3 - Edicion inline:** Al hacer clic en un campo editable, se abre un modal inline del Studio dentro de la misma ventana. Al guardar, los cambios se publican y la pagina se revalida automaticamente via defineLive, mostrando el contenido actualizado sin recarga manual.

Importante: El footer credit "Desarrollado por FastPagePro.com" no sera detectado como editable porque no proviene de ninguna query con Stega - es una string literal hardcodeada.

9.1 Configuracion en layout.tsx:

```
import { VisualEditing } from '@components/VisualEditing';
import { SanityLive } from '@sanity/live';
// En RootLayout:
<SanityLive>{children}</SanityLive>
{process.env.NODE_ENV === 'development' && <VisualEditing />}
```

10. Funcionamiento de Presentation Tool

El Presentation Tool reemplaza Vision/Releases y consiste en dos tabs: Structure (Studio normal) y Presentation (preview en tiempo real). Al modificar un documento en Structure, se puede cambiar a Presentation para ver el contenido en el sitio web real sin abrir otra ventana.

- **Studio Route:** La ubicacion del Studio se configura en sanity.config.ts con basePath: "/admin". Esta ruta resuelve las ubicaciones de vista previa del Presentation Tool.

- **Preview Locations:** Las preview locations mapean cada tipo de documento a una URL del sitio. Un documento "project" con slug "mi-proyecto" se previsualiza en /proyectos/mi-proyecto.
- **Draft Mode Integration:** Los API routes /api/draft-mode/enable y disable gestionan cookies para activar la consulta de documentos draft. El Presentation Tool habilita Draft Mode automaticamente al abrir una preview.
- **Exclusion del Footer Credit:** El credito del footer nunca aparecera como editable en Presentation Tool porque: no esta en ningun documento de Sanity, no aparece en queries GROQ, no tiene source maps Stega, y las preview locations solo aplican a documentos CMS.

11. Manejo de Imagenes, Videos y Multimedia

11.1 Imagenes

Todas las imagenes se almacenan como assets de Sanity y se sirven via `cdn.sanity.io`. `@sanity/image-url` genera URLs optimizadas con parametros de transformacion en tiempo real.

- **Image URL Builder:** URLs del tipo `cdn.sanity.io/images/{projectId}/{dataset}/{assetId}-{dimensions}.webp` con parametros automaticos de formato WebP, calidad 85, dimensiones segun componente.
- **next.config.ts:** Se agregara `cdn.sanity.io` al array `images.remote` en `next.config.ts` para permitir que el componente `<Image>` de Next.js optimice imagenes de Sanity.
- **Metadata:** Metadata automatico: dimensiones, MIME, tamano, SHA1 hash, paleta de colores dominante. Las queries GROQ pueden proyectar estos metadatos para lazy loading y placeholders blur.

11.2 Videos (Hero Slides)

Los videos se almacenan como assets con tipos `video/mp4` y `video/webm`. Se sirven en formato original (sin transformacion CDN). En escritorio, si existe `videoMP4/WebM` se reproduce como fondo con `muted/autoplay/loop`. En movil, siempre se muestra la imagen de fallback para ahorrar datos. La `posterImage` se muestra mientras el video carga.

11.3 Galerias de Proyectos

Cada proyecto soporta hasta 15 imagenes en galeria. El lightbox existente (zoom, pan, swipe, rotacion) se mantiene intacto; solo se reemplazan las URLs de Unsplash por URLs de Sanity. No se requiere modificacion en la logica del lightbox.

12. Estrategia de Cache e ISR

Tres pilares: SSG con revalidacion periodica (ISR), revalidacion bajo demanda via defineLive, y cache del CDN de Vercel. Tiempos de carga rapidos sin sacrificar actualizacion en tiempo real.

- **ISR (Incremental Static Regeneration):** Paginas generadas estaticamente en build, revalidadas cada 60 segundos (revalidate: 60). Contenido actualizado max en 60s sin rebuild.
- **defineLive (Tiempo Real):** Conexion WebSocket con Sanity que escucha mutaciones. Cuando un documento cambia, dispara revalidacion inmediata. Latencia reducida de 60s a casi instantaneo.
- **CDN de Vercel:** HTML cacheado en edge nodes globales. Revalidacion invalida cache automaticamente. Stale-while-revalidate garantiza que el sitio nunca sea lento.
- **Sanity CDN (Imagenes):** Cache agresivo (1 ano). URLs unicas por dimensiones y hash del asset. Actualizar imagen en Sanity genera nueva URL, invalidando cache automaticamente.
- **Draft Mode (Sin Cache):** Con Draft Mode habilitado, queries se ejecutan con token de preview sin cache. Cambios visibles en tiempo real. Se deshabilita al cerrar navegador.

13. Compatibilidad con Vercel

La integracion esta disenada para Vercel. Next.js 16 con App Router es el runtime nativo, y todos los sistemas de Sanity estan probados y compatibles con la plataforma.

- **Studio Embebido:** NextStudio de next-sanity se ejecuta como Server Component en edge sin servidor dedicado.
- **Environment Variables:** Se configuran en Vercel dashboard. NEXT_PUBLIC_ se inyectan en build; las demas son server-only.
- **ISR y Revalidacion:** Vercel soporta ISR nativo con revalidate. 60s optimo para sitios corporativos.
- **Edge Functions:** API routes de Draft Mode se ejecutan como Edge Functions, rapidas y globales.
- **Imagenes Optimizadas:** <Image> de Next.js con cdn.sanity.io permite optimizacion desde infrastructure Vercel (hasta 80% menor).
- **Deploy Automatico:** Push a main dispara deploy. Build time tipico: 45-90 segundos.

14. Compatibilidad con Clonaciones FastPagePro

Arquitectura como plantilla maestra reutilizable. Al clonar para nuevo proyecto, solo cambios de configuracion: no se modifican schemas, queries ni componentes.

14.1 Checklist de Clonacion:

- Actualizar NEXT_PUBLIC_SANITY_PROJECT_ID con ID del nuevo proyecto
- Actualizar NEXT_PUBLIC_SITE_URL con el dominio nuevo
- Actualizar SANITY_API_READ_TOKEN y SANITY_API_TOKEN_PREVIEW
- Modificar sanity/lib/constants.ts: BRAND_COLORS, COMPANY_NAME, STUDIO_TITLE, SITE_URL
- Cambiar logo, favicon y OG images en public/ y Sanity
- Actualizar contenido del CMS desde Studio (no tocar código)
- Configurar dominio personalizado en Vercel
- El crédito "Desarrollado por FastPagePro.com" permanecerá automáticamente en el footer

14.2 Verticales Soportadas:

Vertical	Aplicacion
Construccion / Ingenieria	Proyectos de construccion, servicios, galeria renders, equipo, testimonios
Arquitectura	Proyectos arquitectonicos, diseno, renders, equipo, estadísticas m2
Logistica / Transporte	Servicios logísticos, flota, cobertura rutas, clientes, métricas entregas
Industria / Manufactura	Productos industriales, líneas producción, certificaciones, capacidad
Corporativo General	Páginas informativas, directorio, servicios, casos de éxito, valores
Tecnologia / Software	Productos digitales, desarrollo, equipo tech, stack, métricas rendimiento

15. Confirmaciones Obligatorias

Lista de verificación final con todas las confirmaciones explícitas solicitadas. Cada ítem está marcado como afirmativo [YES] o negativo [NO]. Estos estados son vinculantes.

- [YES] Textos editables - Todos los textos públicos serán editables desde Sanity Studio
- [YES] Imágenes editables - Todas las imágenes almacenadas y editables desde Sanity
- [YES] Videos editables - Videos de hero slides (MP4/WebM) gestionados desde Sanity
- [YES] Hero Video editable desde CMS - Imagen + video MP4 + WebM + poster + fallback móvil, todo configurable
- [YES] Open Graph editable - Título, descripción, imagen, URL canónica desde siteSettings del CMS
- [YES] SEO editable - Title tag, meta description, URL canónica dinámicos desde CMS
- [YES] Proyectos editables - Título, cliente, año, descripción (Portable Text), galería, tags, estado, video
- [YES] Servicios editables - Título, subservicios, categoría, imagen, orden, destacado
- [YES] Testimonios editables - Nombre, rol, quote, rating, avatar, destacado
- [YES] Equipo editable - Nombre, rol, foto, bio (Portable Text), email, redes, orden
- [YES] Configuración global editable - Contacto, redes, ubicación, SEO, OG desde siteSettings
- [YES] Visual Editing funcional - Overlay de edición inline sobre el sitio en producción
- [YES] Presentation Tool funcional - Tabs Structure y Presentation con Draft Mode
- [NO] El crédito "Desarrollado por FastPagePro.com" permanecerá HARDCODEADO y fuera del CMS permanentemente

16. Plan de Implementación (5 Fases)

- **Fase 1: Infraestructura:** Verificar sanity.config.ts, schemas (10 tipos), constants.ts, schema-master.ts. Validar compilación en Studio /admin.
- **Fase 2: Cliente y Queries:** Verificar sanity.client.ts (Stega), sanity.queries.ts (10 queries), sanity/live.ts (defineLive). Verificar API routes Draft Mode.
- **Fase 3: Studio Route:** Verificar src/app/admin/[...tool]/page.tsx. Validar Studio carga correctamente con 10 tipos de documento.
- **Fase 4: Layout:** Modificar layout.tsx: SanityLive + VisualEditing. Modificar next.config.ts: cdn.sanity.io en images.remote.
- **Fase 5: Migración Frontend:** Migrar componentes en orden: Footer > ServicesPage > ProjectsPage > HomePage. Cada uno con fallback hardcodeado.

Documento generado por FastPagePro.com - Arquitectura de Plantilla Maestra v1.0
<https://sertrade.vercel.app>